* Pumping Lemma for Regular Languages

* Let us consider the language

$$L_{01} = \{ 0^n 1^n \mid n \geq 1 \}$$

* This language contains all strings 01, 0011, 000111, and so on, that consist of one or more 0's followed by an equal number of 1's.

$$L_{01} = \{ 01, 0011, 000111, \ldots\ldots \}$$

* We claim that L_{01} is not a regular language.
* The intuitive argument is that if L_{01} were regular, then L_{01} would be accepted by some DFA A.
* Now, A should have some finite number of states, say k states.
* Now assume that, automaton A is receiving k 0's as i/p.
* Now, A will go to some state after consuming each of the (k+1) prefixes of the i/p: ϵ, 0, 00, ..., 0^k.

$$\overbrace{\underbrace{\epsilon}\,\underbrace{0\,0\,0\,0}\,0\,0\,0

Pigeon hole Principle

* If we put n objects into m boxes (pigeonholes), and if $n > m$, then at least one box must have more than one item in it.

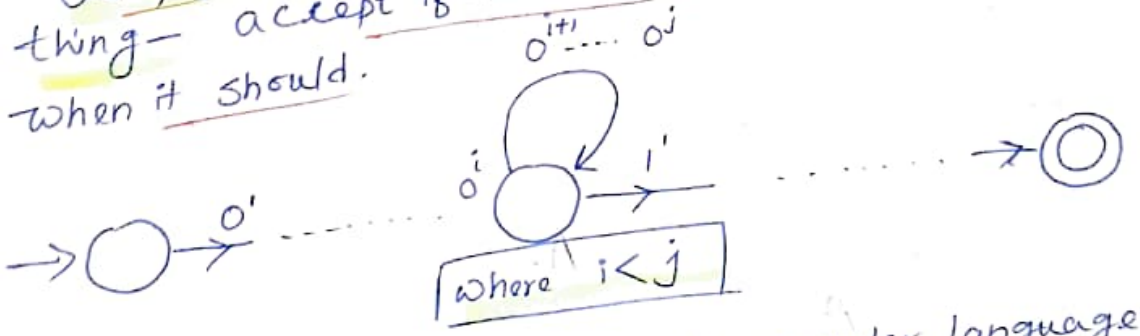
---

* Now, there are k different states in automaton **A**, the pigeonhole principle tells us that after reading two different prefixes, say 0^i and 0^j, **A** must be in the same state, say q.

* However, suppose instead that automaton **A**, after reading i or j 0's starts receiving 1's as input.

* After receiving i 1's, it must accept it (should go to the final state) if it previously received i 0's, but not if it received j 0's.

* Since it was in state q when the 1's started, it can't "remember" whether it received i or j 0's, so we can fool **A** and make it do the wrong thing — accept if it should not, or fail to accept when it should.



Theorem (The pumping lemma for regular languages)
Let L be a regular language. Then there exists a constant n (which depends on L) such that for every string $w \in L$ with $|w| \geq n$, we can break w into three strings, $w = xyz$, such that

→ no. of states in the DFA that recognizes L, n is also called the pumping length, all strings longer than n can be pumped.

1. $\boxed{y \neq \epsilon \text{ or } |y| \geq 1}$
2. $\boxed{|xy| \leq n}$
3. For all $k \geq 0$, the string $w_k = xy^kz$ is also in L.

* That is, we can always find a non-empty string y not too far from the beginning of w that can be "pumped", that is repeating y any number of times, or deleting it ($k=0$), keeps the resulting string in the language L.

Proof:

Suppose L is regular. Then there exists a DFA A that recognizes it.

Suppose A has n states. Now, consider any string w with $|w| \geq n$, say $w = a_1a_2 \ldots\ldots a_m$, where $m \geq n$ and each a_i is an i/p symbol.

For $i = 0, 1, \ldots, n$ define state p_i to be $\delta^*(q_0, a_1a_2 \ldots a_i)$ where δ^* is the extended transition function of A, and q_0 is the start/initial state of A (Note that $p_0 = q_0$)

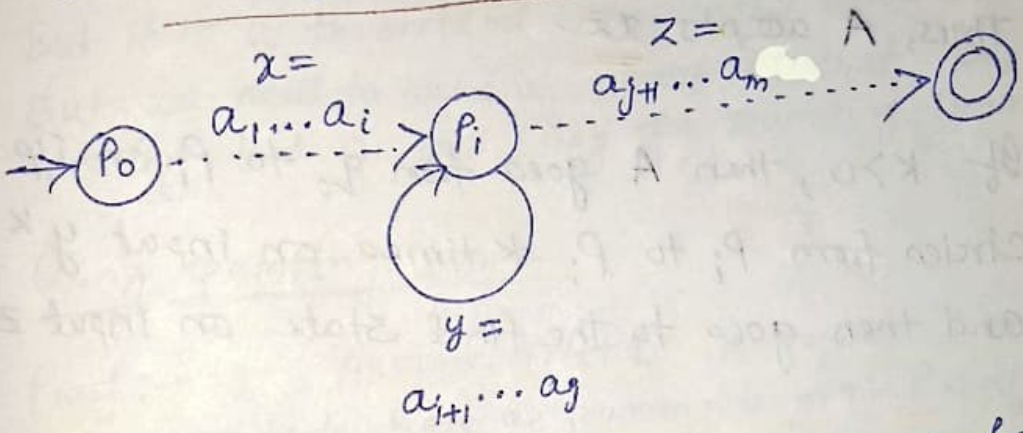
p_i is the state after reading the first i symbols of w.

By the pigeonhole principle, it is not possible for the $n+1$ different p_i's for $i = 0, 1, \ldots, n$ to be distinct, since there are only n different states.

✓ Thus we can find two different integers i and j, with $\boxed{0 \le i < j \le n}$, such that $P_i = P_j$.

✓ Now we can break $w = xyz$ as follows:

1. $x = a_1 a_2 \cdots a_i$
2. $y = a_{i+1} a_{i+2} \cdots a_j$
3. $z = a_{j+1} a_{j+2} \cdots a_m$



✓ That is, x takes us to P_i once; y takes us from P_i back to P_i (since P_i is also an P_j), and z is the balance of w.

✓ Note that x may be empty, in the case that $i=0$, Also z may be empty if $j = n = m$.

✓ However, y can not be empty, since i is strictly less than j $\therefore$

$\boxed{i < j}$

* Now consider what happens if the automaton A receives the i/p xy^kz for any $k \geq 0$.

- If $k=0$, then A goes from the initial state q_0 (which is also P_0) to P_i on input x. Since P_i is also P_j it must be that A goes from P_i to the final state on input z.

  Thus, A accepts xz.

- If $k>0$, then A goes from q_0 to P_i on i/p x, circles from P_i to P_i k times on input y^k, and then goes to the final state on input z.

* Thus for any $k \geq 0$, xy^kz is also accepted by A; that is, xy^kz is in L.

* Using pumping lemma prove that $L = \{a^n b^n \mid n \geq 1\}$ is not regular.

⇒ Intuitively, we can say why it is not regular, because for this language we need to keep a count of how manny a's we have and then we repeat that many b's.

But a finite state machine (FSM) can not keep count of infinite length of strings.

So, this can not be designed by FSM.

But it is a theoritical knowledge/ intuition.

But we need to have a proof for that. So, for proving that we will using the pumping lemma.

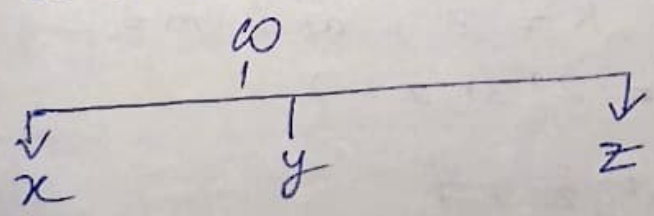
Using pumping lemma

Proof:- Assume that L is regular. Now, it needs to have a puming length P such that any string whose length is greater than or equal to P would be able to be pumped.

✓ Now let us choose the string $\omega \in L$ with $|\omega| \geq P$

$$\omega = a^P b^P$$

✓ Now let us see how can we decompose



Such that $|xy| \leq P$

S.O, xy portion will occur in a part

$$\omega = \underbrace{aaaa}_{x}\underbrace{|\cdots|}_{y}\underbrace{aabbbb\ldots\ldots b}_{z}$$

* Suppose

$$y = a^k$$

and $x = a^l$

and $z = a^m b^p$

So,

$$l + k + m = p$$

So, ω is in the following form

$$a^l (a^k) a^m b^p$$

You can pump this portion. So, you will get

$$a^l (a^k)^i a^m b^p \in L \quad \forall i \geq 0$$

$i = 0, 1, 2, \ldots$

So, you can get so many strings where no. of a's is not equal to no. of b's

if $l = 2$, $k = 3$, and $m = 2$

So, $p = 2 + 3 + 2 = 7$

$$a^2 (a^3)^i a^2 b^7$$

$$\Rightarrow \underbrace{aa}_{x} \underbrace{aaa\,aaa}_{y^2} \underbrace{aabbbbbb}_{z} \quad i = 2$$

where $n_a(\omega) = 10$
$n_b(\omega) = 7$] not equal

Therefore such a string doesnot belong to the language $L = \{ a^n b^n \mid n \geq 1 \}$

This contradicts the pumping lemma and thereby indicates that the assumption that L is regular must be false.

---

* Using pumping lemma prove that $L = \{ a^{n^2} \mid n \geq 1 \}$ is not regular.

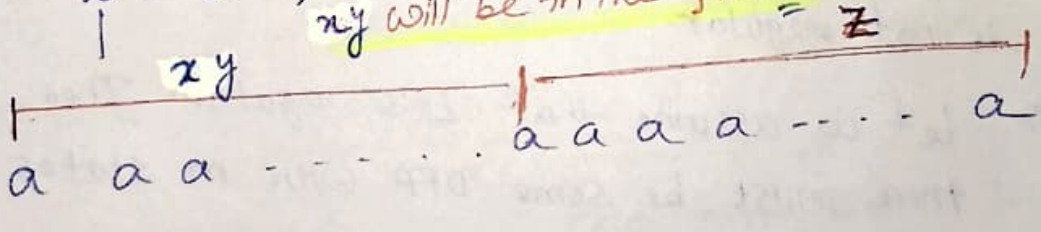
$\Rightarrow$ Suppose L is regular. Then L will be accepted by a FSM (DFA) with p states.

By pumping lemma there is a pumping length p such that any string whose length is $\geq p$ would be able to be pumped.

✓ Now let us choose the string $\omega \in L$ with $|\omega| \geq p$

$$\omega = a^{p^2}, \quad |\omega| = \underline{p^2 > p}$$

xy will be in the first p a's.



$|xy| \leq p \qquad |y| \geq 1$

So, $\quad 1 \leq |y| \leq p$

So, if you consider $x y^2 z$ then what can you say about the length

$$|xy^2z| = |xyz| + |y|$$

$$\leq p^2 + p$$ (which is greater than p^2) — not a square.

But after p^2, the next string you get is of length

$(p+1)^2$, i.e., $\omega = a^{(p+1)^2}$

$$L = \{a^{1^2}, a^{2^2}, a^{3^2}, a^{4^2}, \ldots\ a^{p^2}, a^{(p+1)^2}, \ldots\}$$

$$= \{a^1, a^4, a^9, a^{16}, \ldots\ \underbrace{a^{p^2}, a^{(p+1)^2}}, \ldots\}$$

By the argument we are getting the string in between a^{p^2} and $a^{(p+1)^2}$ which is not correct.

So, we are arriving at a contradiction.

So, L is not regular.

---

* Show that the Language (using pumping lemma)

$$L = \{a^n b^k c^n : n \geq 0, k \geq 0\}$$

is not regular

$\Rightarrow$ Let us assume that L is regular. Then there must be some DFA with m states that recognizes L.

By pumping lemma, there must be a pumping length m such that strings $\omega \in L$ with $|\omega| \geq m$ would be able to be pumped.

Pick $\omega = \underline{a^m b c^m} \in L$, $|\omega| = 2m+1 \geq m$

Now, $|xy| \leq m$ so xy will be chosen from a's only.

aaa - - - - |aa| b ccc - - - - cc.

$\leftarrow |xy| \rightarrow$

$\leftarrow \quad m \quad \rightarrow$

$|xy| \leq m, |y| \geq 1$ So,

$$1 \leq |y| \leq m$$

Suppose $y = a^p$

$x = a^q$

$z = a^r b c^m \Rightarrow \boxed{q + p + r = m}$

$$w_i = a^q (a^p)^i a^r b c^m$$

$$w_2 = a^q a^{2p} a^r b c^m$$

$m + p > m$ $\quad n_a(w) > n_c(w)$

Now, $q + 2p + r \neq m$

So, $w_2 \notin L$, This contradiction implies that L is not regular language.

P-174

* Using pumping Lemma show that

$L = \{a^n b^k c^n : n \geq 0, k \geq n\}$ is not regular.

$\Rightarrow$ Let us assume that L is regular. Then there must be a DFA that recognizes L (with m states)

By pumping lemma, there must be a pumping length 'm' such that

$w \in L$ with $|w| \geq m$ can be decomposed into $w = \underline{x}\,\underline{y}\,\underline{z}$

Pick $w = \underline{a}^m \underline{b}^m \underline{c}^m$ (for b's you can take value $\geq m$)

$|w| = 3m > m$

Now, $|y| \geq 1$

$|xy| \leq m$

so, xy portion will be choosen from a's only.

$$\overbrace{aaaaa\ldots a}^{m\ a's}\overbrace{bbbb\ldots b}^{m\ b's}\overbrace{ccc\ldots c}^{m\ c's}$$

$$\underbrace{aaaaa\ldots a}_{x}\,\underbrace{bbbb\ldots b}_{y}\,\underbrace{ccc\ldots c}_{z}$$

Let us assume,

$x = a^p$ where $p+q+r = m$

$y = a^q$

and $z = a^r b^m c^m$

Now, repeat (pump) y part.

$$\omega = xyz$$
$$= \underbrace{a^p}_{x}\underbrace{a^q}_{y}\underbrace{a^r b^m c^m}_{z} \quad \text{where } p+q+r=m$$

Now, $\omega_2 = a^p (a^q)^2 a^r b^m c^m$

$$= a^p a^{2q} a^r b^m c^m$$
$$= a^{p+2q+r} b^m c^m$$

Here, no. of a's

$$= p+2q+r$$
$$= p+q+r+q$$
$$= m+q > m$$

Here $n_a(\omega) > n_b(\omega)$

$n_a(\omega) > n_c(\omega)$

But according to the Language L

$$n_a(\omega) = n_c(\omega)$$

and $n_b(\omega) \geq n_a(\omega) = n_c(\omega)$

Hence, we are arriving at a contradiction.

Hence L is not regular.

P-176

* Prove that $L = \{a^n b^l c^k : k \le n+l\}$ is not regular using pumping lemma (PL).

$\Rightarrow$ Suppose that L is regular and by pumping lemma there must be a pumping length 'm' such that $w \in L$ with $|w| \ge m$ can be pumped.

Pick up $w = a^m b^m c^{2m}$

$|w| = 2m + 2m$
$= 4m > m$

Now w can be decomposed as
$w = xyz$; $|y| \ge 1$
$|xy| \le m$

So, xy portion will be choosen from a's only

$$\overbrace{aa\ldots\ldots a\ldots a}^{m\ a's}\overbrace{bbb\ldots b}^{m\ b's}\overbrace{ccc\ldots c}^{2m\ c's}$$

$\leftarrow x \rightarrow | y | \leftarrow z \rightarrow$

Suppose
$x = a^p$
$y = a^q$
$z = a^r b^m c^{2m}$

So, $w = xyz$
$= a^p a^q a^r b^m c^{2m}$ where $p+q+r = m$

$k \le n+l$

$k = n+l$

$k = m+m$
$= 2m$

Take every index in terms of m so that it would be convenient to compare the length of $w \in L$ with pumping length 'm'

You can also pickup
$w = a^m b^{m+2} c^{2m}$

$k \le m+m+2$

$k < 2m+2$

$k = 2m$

Now, $w_0 = a^p (a^q)^0 a^r b^m c^{2m}$

$= a^p a^r b^m c^{2m}$

$= a^{p+r} b^m c^{2m}$

Now,

$n_a(w) = p+r < m$

$n_b(w) = m$

So, $n_a(w) + n_b(w) = p+r+m < \underset{n_c(w)}{2m}$

where $n_c(w) = \underline{\underline{2m}}$, i.e, $k > n+l$

$n_c(w) > n_a(w) + n_b(w)$

However, according to L,

$n_c(w) \leq n_a(w) + n_b(w)$

Hence, we are arriving at a contradiction. Hence L is not regular.

(*) $L = \{a^n b^l c^k : k \neq n+l\}$

$\Rightarrow$ Suppose that L is regular and by pumping Lemma there must be a pumping length m such that $w \in L$ with $|w| \geq m$ can be pumped.

Pick up $w = a^m b^m c^{3m}$ | Here, $3m \neq 2m$ $= m+m$

$|w| = 3m > m$

Now, $w = xyz$ $\quad |y| \geq 1, |xy| \leq m$

So, $w = \underset{x}{a^p} \underset{y}{a^q} \underset{z}{a^r b^m c^{3m}}$

P-178

$w_i = xy^i z$

Pump y as many times as you want even eliminate y by considering $w_0 = xy^0 z$

However, you can't prove (That is our assumption to pick $w = a^m b^m c^{3m}$ is wrong)

$$n_a(w) + n_b(w) = n_c(w)$$

$$\boxed{n + l = k}$$

* Our aim is to prove that L is not regular.

How to do that?

$\Rightarrow$ First try to understand how the w is being pumped.

$$w = \underline{\underline{x}}\ \underline{\underline{y}}\ \underline{\underline{z}}$$

where $x = a^p$, $y = a^q$, $z = a^r b^m c^{3m}$ (not bothering)

$$= a^p a^q a^r b^m c^{3m}$$

$$= a^{p+q+r} b^m c^{3m}$$

where $p + q + r = m$

Now for

$$w_1 = a^{p+q+r} b^m c^{3m}$$

$$w_2 = a^p (a^q)^2 a^r b^m c^{3m}$$

$$= a^p a^{2q} a^r b^m c^{3m}$$

$$= a^{p+2q+r} b^m c^{3m}$$

$$= a^{m+q} b^m c^{3m}$$

$$= a^{m+(2-1)q} b^m c^{3m}$$

$$w_3 = a^p (a^q)^3 a^r b^m c^{3m}$$
$$= a^{p+3q+r} b^m c^{3m}$$
$$= a^{\underline{p+q+r}+2q} b^m c^{3m}$$
$$= a^{m+2q} b^m c^{3m}$$
$$= a^{m+(3-1)q} b^m c^{3m}$$

$\vdots$

$$w_i = a^{m+(i-1)q} b^m c^{3m}$$

---

* Now, $L = \{a^n b^l c^k : k \neq n+l\}$

Somehow we have to prove for

Some w_i, $\boxed{k = n+l}$ $\quad (n_a(w) + n_b(w) = n_c(w))$

But y is a portion of a^m

i.e., $y = a^q$, Take $w = a^m b^m c^{k'}$

If we repeat y many times like $(i+1)$ times

then $w_{i+1} = a^{m+iq} b^m c^{k'}$

Here, $n_a(w) = m+iq$

$n_b(w) = m$

$n_a(w) + n_b(w) = 2m + iq$

Now we can't say

$(k')\, n_c(w) = n_a(w) = n_b(w)$

as m is not divisible by q or q is not related to m.

So,

pick up $\omega = a^{m!}\, b^{m!}\, c^{2(m+1)!}$

Now,

$$|\omega| = m! + m! + 2(m+1)!$$
$$= 2m! + 2(m+1)! > m$$

Now,

$$m! = \underbrace{1 \times 2 \times 3 \times 4 \times \cdots \times q \times (q+1) \times \cdots \times m}$$

So, q is a factor of $m!$

$m!$ is divisible by q

Hence

$$\omega = a^{m!}\, b^{m!}\, c^{2(m+1)!}$$

So, $\omega_i = a^{m! + (i-1)q}\, b^{m!}\, c^{2(m+1)!}$

Now,

$$\omega_{\frac{2mm!}{q}+1} = a^{m! + \left[\frac{2mm!}{q} + 1 - 1\right] q}\, b^{m!}\, c^{2(m+1)!}$$

$$= a^{m! + \frac{2mm!}{q} \times q}\, b^{m!}\, c^{2(m+1)!}$$

$$= a^{m! + 2mm!}\, b^{m!}\, c^{2(m+1)!}$$

$$= a^{2mm! + m!}\, b^{m!}\, c^{2(m+1)!}$$

$$h_a(\omega) + h_b(\omega) = 2mm! + m! + m!$$
$$= 2mm! + 2m!$$

$$= 2m!(m+1) = 2(m+1)!$$

$$= n_c(w)$$

Therefore L is not regular.

---

* Properties of Regular languages (RLs)

* Closure properties of Regular Languages

What do we mean by closed under operation?

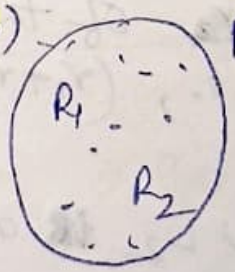
If we take a set of all natural numbers $\mathbb{N} = \{1, 2, 3, 4, \ldots\}$ then we can say that $\mathbb{N}$ is closed under addition ('+'), i.e, if we take any two natural numbers from $\mathbb{N}$ and if we apply the addition ('+'), then the resulting natural number also belong to $\mathbb{N}$

$\mathbb{N}$ is closed under '+'

$(n_1 + n_2) \in \mathbb{N}$

where $n_1, n_2 \in \mathbb{N}$

Similarly, Whenever we take the set of all regular languages and if we apply some operation (OP) on any two regular languages (RLs) and if the resulting language is also regular then we can say that $\mathbb{R}eg$ (Set of all RLs) is closed under OP.



$\mathbb{R}eg$. (Set of all RLs)

$(R_1 \text{ OP } R_2) \in \mathbb{R}eg$

$R_1, R_2 \in \mathbb{R}eg$.

or, Regular Languages are closed under some operation OP.